```cpp
#include "VigenereIterator.h"
#include <cctype>   // for isalpha, toupper, tolower

// Constructor to set up the iterator using a member initializer list and then prepares the mapping table
// if a source text exists, it processes the first character appropriately (encode or decode based on the mode)
VigenereIterator::VigenereIterator(const std::string& aKeyword,
                                   const std::string& aSource,
                                   EVigenereMode aMode) noexcept
    : fMode(aMode),
      fKeys(aKeyword),
      fSource(aSource),
      fIndex(0),
      fCurrentChar('\0')
{
    // Set up the cipher mapping using the evaluator's definition
    initializeTable();

    // Process the first character from fSource if available
    if (!fSource.empty()) {
        if (fMode == EVigenereMode::Encode)
            encodeCurrentChar();
        else
            decodeCurrentChar();
    }
}

// This function enciphers the character at the current position in fSource and only processes alphabetic characters
// The non alphabetic characters are passed unchanged
// The function first determines whether the letter was lowercase and then converts it to uppercase for processing.
// It uses the current auto key character to lookup the encoded result from fMappingTable,
// restores the letter's case if necessary, and stores the
```

```cpp
 result in fCurrentChar.
// Finally, it advances the auto key iterator and then appends the processed letter to the key.
void VigenereIterator::encodeCurrentChar() noexcept {
    if (fIndex >= fSource.size()) {
        fCurrentChar = '\0';
        return;
    }

    char current = fSource[fIndex];

    // Non alphabetic characters are not processed by the cipher
    if (!std::isalpha(static_cast<unsigned char>(current))) {
        fCurrentChar = current;
        return;
    }

    // Save whether the current character is lowercase for later restoration
    bool isLower = std::islower(static_cast<unsigned char>(current));
    // Convert the letter to uppercase for cipher processing
    char plainChar = static_cast<char>(std::toupper(static_cast<unsigned char>(current)));

    // Retrieve the current letter from the auto key
    char keyChar = *fKeys;
    int row = keyChar - 'A';
    int col = plainChar - 'A';

    // Find the ciphered character using the mapping table
    char encoded = fMappingTable[row][col];

    // If the original was lowercase, adjust the result accordingly
    if (isLower)
        encoded = static_cast<char>(std::tolower(static_cast<unsigned char>(encoded)));
    fCurrentChar = encoded;

    // Advance the auto key iterator and append the
```

```
65 uppercase letter to it
66     ++fKeys;
67     fKeys += plainChar;
68 }
69
70 // This function deciphers the character at the current
   position in fSource and only processes alphabetic
   characters.
71 // Non alphabetic characters are passed unchanged.
72 // Alphabetic characters are converted to uppercase and
   decoded using the auto key row in fMappingTable,
73 // Then they are restored to their original case.
74 // Finally the auto key is advanced and updated with the
   decoded letter.
75 void VigenereIterator::decodeCurrentChar() noexcept {
76     if (fIndex >= fSource.size()) {
77         fCurrentChar = '\0';
78         return;
79     }
80
81     // Get the current character from fSource
82     char current = fSource[fIndex];
83
84     // Pass through non alphabetic characters
85     if (!std::isalpha(static_cast<unsigned char>(current
   ))) {
86         fCurrentChar = current;
87         return;
88     }
89
90     // Preserve the original case of the character
91     bool isLower = std::islower(static_cast<unsigned char
   >(current));
92
93     // Convert for processing
94     char encodedChar = static_cast<char>(std::toupper(
   static_cast<unsigned char>(current)));
95
96     // Get the current key character
97     char keyChar = *fKeys;
98     int row = keyChar - 'A';
99
100    // Perform a linear search through the row to find
   the matching character
```

```
101     int col = 0;
102     for (; col < CHARACTERS; col++) {
103         if (fMappingTable[row][col] == encodedChar)
104             break;
105     }
106     char decoded = static_cast<char>('A' + col);
107     if (isLower)
108         decoded = static_cast<char>(std::tolower(
    static_cast<unsigned char>(decoded)));
109     fCurrentChar = decoded;
110 
111     // Advance and update the auto key with the
    deciphered letter
112     ++fKeys;
113     fKeys += static_cast<char>(std::toupper(decoded));
114 }
115 
116 // Returns the current ciphered or deciphered character
117 char VigenereIterator::operator*() const noexcept {
118     return fCurrentChar;
119 }
120 
121 // Moves the iterator forward by incrementing fIndex,
122 // then processes the next character using either
    encoding or decoding
123 VigenereIterator& VigenereIterator::operator++() noexcept
     {
124     fIndex++;  // Move to the next character in fSource
125     if (fIndex < fSource.size()) {
126         if (fMode == EVigenereMode::Encode)
127             encodeCurrentChar();
128         else
129             decodeCurrentChar();
130     } else {
131         // Signal the end
132         fCurrentChar = '\0';
133     }
134     return *this;
135 }
136 
137 // Returns a copy of the iterator before incrementing,
    then advances the iterator
138 VigenereIterator VigenereIterator::operator++(int)
    noexcept {
```

```
139     VigenereIterator temp = *this;
140     ++(*this);
141     return temp;
142 }
143 
144 // Two iterators are considered equal if they point to
    the same position in the same source
145 bool VigenereIterator::operator==(const VigenereIterator
    & aOther) const noexcept {
146     return (fIndex == aOther.fIndex &&
147             fSource == aOther.fSource);
148 }
149 
150 // Returns an iterator reset to the start of the source
151 VigenereIterator VigenereIterator::begin() const noexcept
     {
152     VigenereIterator iter = *this;
153     iter.fKeys.reset();
154     iter.fIndex = 0;
155     if (!iter.fSource.empty()) {
156         if (iter.fMode == EVigenereMode::Encode)
157             iter.encodeCurrentChar();
158         else
159             iter.decodeCurrentChar();
160     }
161     return iter;
162 }
163 
164 // Returns an iterator positioned immediately after the
    last character in the source
165 VigenereIterator VigenereIterator::end() const noexcept {
166     VigenereIterator iter = *this;
167     iter.fIndex = fSource.size();
168     iter.fCurrentChar = '\0';
169     return iter;
170 }
171 
```